

Criteria	Novice	Developing	Competent	Proficient
Problem Clarity (10%)	Vague problem scope.	Clear scope but weak justification.	Defined scope with clear objectives.	Clearly defined scope, objectives, and industry relevance.
Domain Integration (5%)	No link to real-world context.	Weak link to real-world application.	Good alignment with target domain.	Strong, innovative application within domain.
Procedural Logic (25%)	Code is unstructured or monolithic.	Procedural constructs used inconsistently.	Robust use of functions and structured control flow.	Efficient, readable, and highly maintainable procedural code.
Functional Definition (10%)	Functions are undefined or poorly named.	Functions lack parameter specifications.	Functions have clear signatures and typed parameters.	Functions follow modularity best practices and handle errors.
Modular Architecture (20%)	Fails to use four-module split.	Modules exist but contain circular dependencies.	Correct modular separation and interface usage.	Highly decoupled modules with clear, single-purpose interfaces.
DevOps Integration (20%)	No version control evidence.	Basic commit history in Git.	Consistent use of branches and pull requests.	Integrated CI/CD workflow with clean commit history and documentation.
Application of Method (10%)	Code does not compile or run as expected.	Code runs with frequent runtime errors.	Code is functional and adheres to structural requirements.	Code is highly robust, well-documented, and error-resilient.